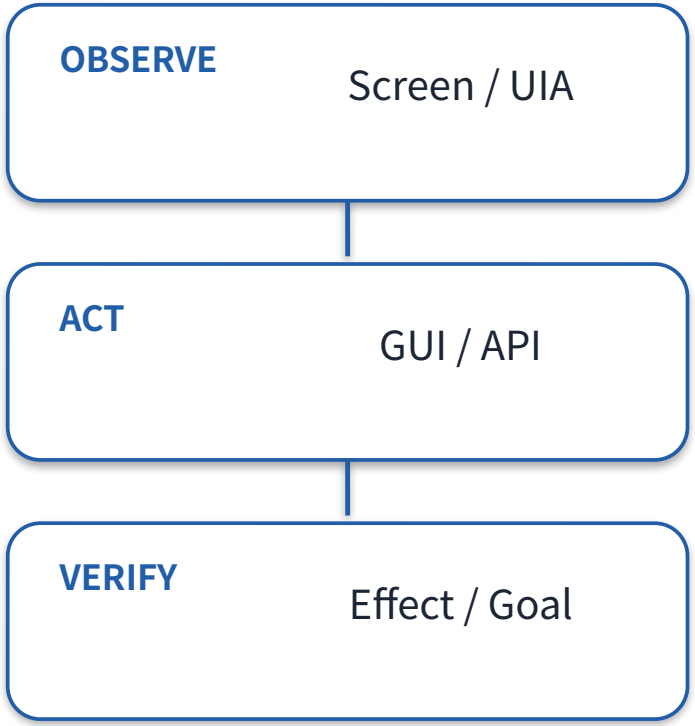
C • WHAT

C． GUI Agent 执行

从可执行环境、 OS 级系统到执行前 / 中 / 后的验证闭环

A 决定何时并行　B 决定在哪里执行　C 决定执行什么
D 决定能否信任



一次错误会改变环境，并沿后续轨迹持续放大

- **离线任务：**输出错误通常只影响当前答案。
- **GUI 任务：** click/type/delete/pay 会改变真实系统状态；错误可能不可逆。

- **额外挑战：**像素 / 控件 grounding 、动态渲染、网络延迟、弹窗、跨应用状态、隐藏系统状态。
- **核心指标：**不能只看单步 action accuracy ，必须看 end-to-end success 、 loop 、 recovery 和副作用。

GUI Agent 是“感知—规划—动作—验证”的完整系统

- **Observation：** screenshot / DOM / a11y / UIA / XML 。
- **Grounding：** element / coordinate / control / text 。
- **Planning：** task → subgoal → candidate actions 。
- **Execution：** GUI event / keyboard / shell / API / MCP 。
- **Verification：** action effect / hidden state / artifact / completion condition 。
- **Recovery：** retry / re-ground / alternative action / rollback / ask user 。

本章从环境与 Actor，走向三阶段 verifier

- **环境底座：** OSWorld 。
- **执行系统：** UFO² 。
- **跨平台 Actor：** Mobile-Agent-v3.5 / GUI-Owl-1.5 。
- **端云移动场景：** CORE 。

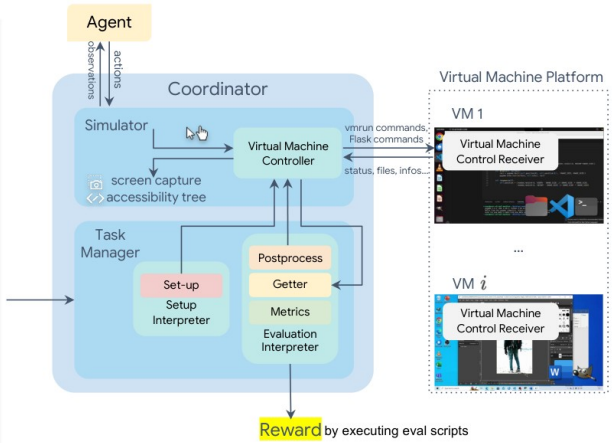
- **执行前：** GUI-Critic-R1 。
- **执行中：** VeriGUI 。
- **执行后：** VAGEN / IRA 。

静态截图数据无法评估替代路径、状态变化与跨应用执行

- **现有问题：**离线 demonstration 只有固定答案；简化 web 环境覆盖面有限；无法进行真实 exploration 与 execution-based evaluation 。
- **OSWorld：**支持 Ubuntu 、 Windows 、 macOS ；允许鼠标、键盘、GUI 与 CLI ；可运行任意应用。
- **每个任务包含：**自然语言指令、初始系统状态 setup 、可复现 VM 、执行脚本 evaluator 。

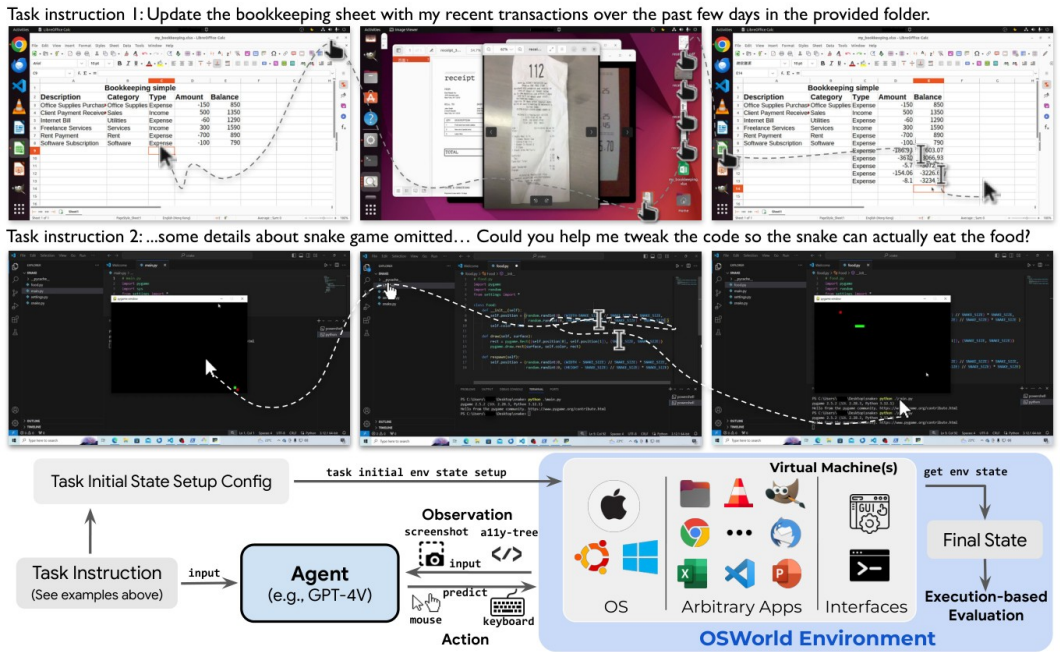
Config

```
{ "instruction": "Please update my bookkeeping sheet with
the recent transactions from the provided folder, detailing
my expenses over the past few days.",
  "config": [{"type": "download",
      "parameters": {"files": [
{"path": "/home/user/Desktop/my_bookkeeping.xlsx",
 "url": "https://drive.google.com/uc?id=xxxx"},
{"path": "/home/user/Desktop/receipt_0.jpeg",
 "url": "https://drive.google.com/uc?id=xxxx"},...]}},
    {"type": "open",
      "parameters": {  "path":
"/home/user/Desktop/my_bookkeeping.xlsx"}}],
   "evaluator": {"postconfig": [{"type": "activate_window",
      "parameters": {"window_name": "my_bookkeeping.xlsx -
LibreOffice Calc",... ],
    "result": {"type": "vm_file",
      "path": "/home/user/Desktop/my_bookkeeping.xlsx",
      "dest": "my_bookkeeping.xlsx"},
    "expected": {"type": "cloud_file",
      "path": "https://drive.google.com/uc?id=xxx",
      "dest": "my_bookkeeping_gold.xlsx" },
    "func": "compare_table",
    "options": {
      "rules": [{
          "type": "sheet_fuzzy",
          "sheet_idx0": "RNSheet1",
          "sheet_idx1": "ENSheet1",
          "rules": [ {"range": [ "A1:A8",... }]}]
}
```



论文原图： OSWorld architecture

369 个开放式计算机任务，覆盖文件、应用与跨应用 workflow



论文原图： OSWorld tasks

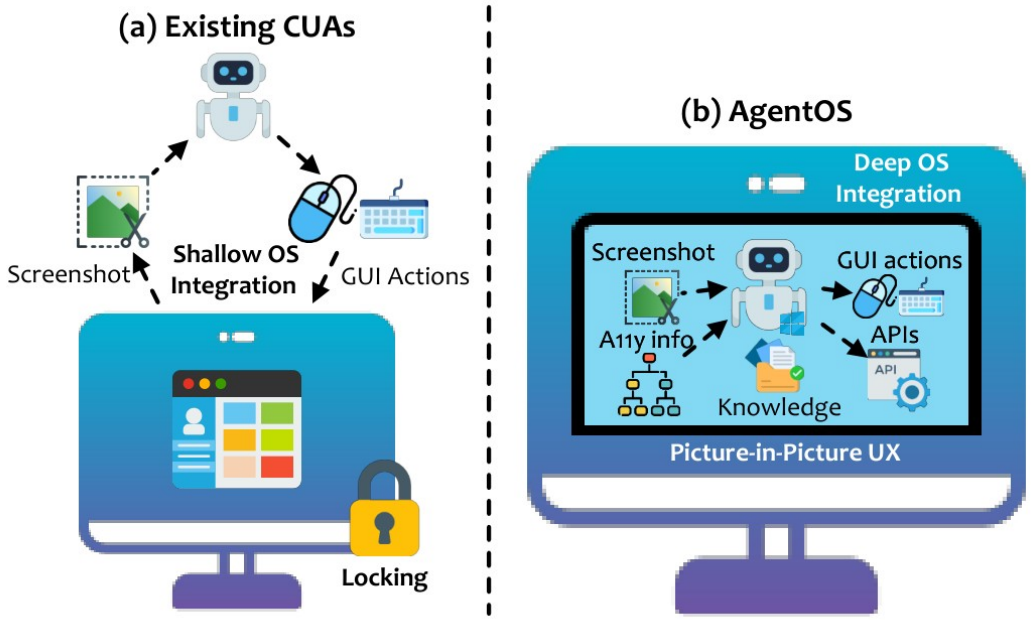
- **任务范围：** 浏览器、办公软件、图像、邮件、代码、OS 文件 I/O 、多应用协同。
- **评测：** 134 类定制 evaluator ，不依赖唯一操作序列，而检查最终环境状态。
- **结果：** 人类完成率 72.36% ； 当时最佳模型仅 12.24% ； 跨应用 workflow 最佳仅 6.57% 。
- **主要错误：** 点击坐标不准、重复动作、弹窗干扰、缺乏 GUI/domain knowledge 。

369	134	72.36%	12.24%
tasks	evaluators	human	then-best model

OSWorld 把 GUI Agent 从“预测动作”推进到“完成任务”

- **价值：**真实可执行、初态可复现、跨 OS/ 跨 app 、支持在线学习。
- **局限 1：**脚本 evaluator 仍可能把替代正确路径判为 false negative 。
- **局限 2：**环境启动与大规模 rollout 成本高。
- **局限 3：** benchmark 本身不提供更可靠的 Actor 架构。
- **过渡：**下一步不是只训练更大的 VLM ，而是把 CUA 变成深度集成 OS 的执行系统。

截图 + 模拟鼠标不足以支撑可扩展桌面自动化

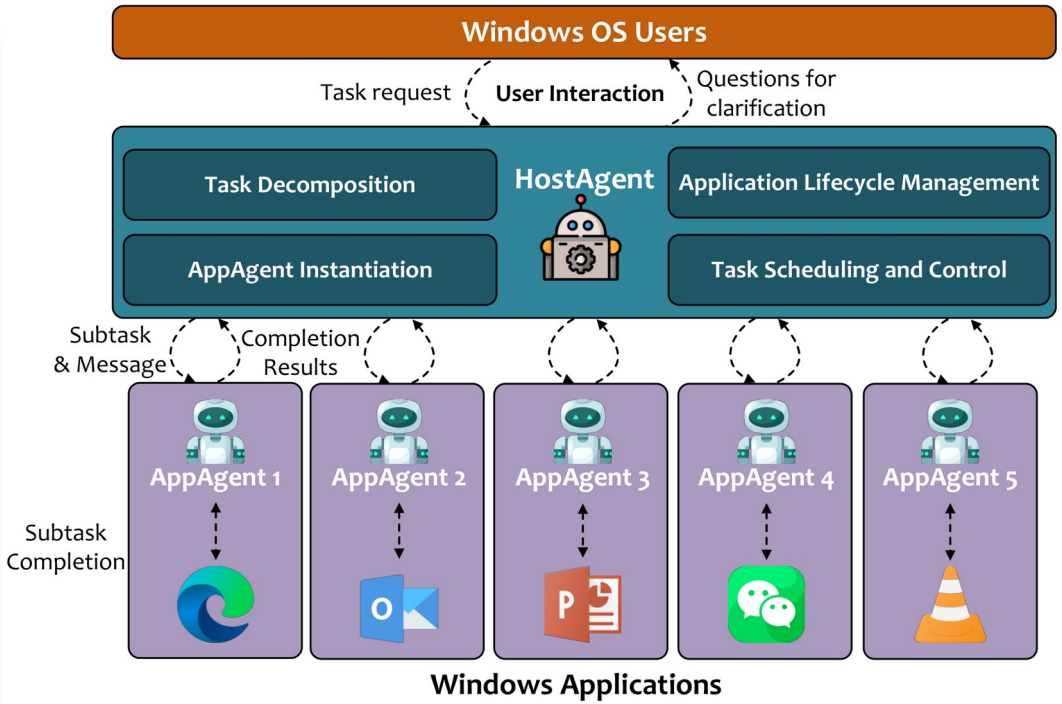


论文原图： UFO2 comparison

- **问题：**视觉输入冗余；缺少 UIA 、应用 API 与 process state ； 自动化占用主桌面并干扰用户。
- **UFO² 定位：** Windows multi-agent AgentOS ， 把 OS 能力、应用专用知识和执行控制嵌入 Agent loop 。
- **核心思想：**不是让一个通用模型直接操作所有应用，而是由中央协调器调度应用专家。

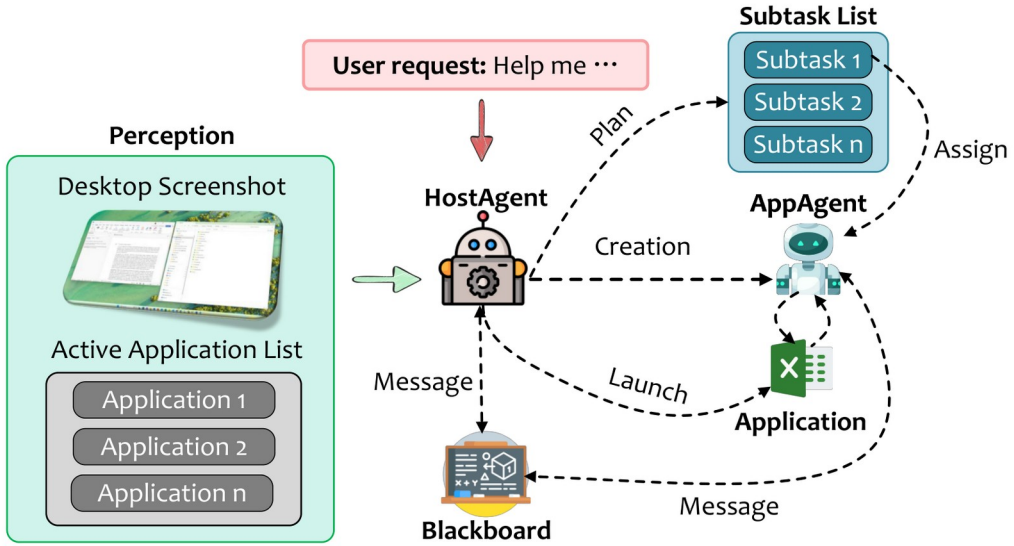
HostAgent 负责跨应用分解， AppAgent 负责应用内可靠执行

- **HostAgent：** 理解用户目标、分解跨应用 subtasks 、选择 / 调度 AppAgent 、汇总进度。
- **AppAgent：** 具备应用特定 API 、 domain knowledge 、 control detection 和 action execution 。
- **Session：** 维护 task 、 application 、 window 、 control 与 execution history 。
- **优势：** 模块化扩展；不同 app 可采用不同 observation/action 接口。



论文原图： UFO2 framework

能调用 API 时不必坚持“像人一样点鼠标”



论文原图： UFO2 HostAgent

- **GUI action：** 适合没有稳定接口、需要视觉定位或低层交互的控件。
- **Native/app API：** 适合结构化、批量、精确且对布局变化鲁棒的操作。
- **UFO²：** 在同一 plan 中混合 GUI 与 API，并由 AppAgent 根据状态选择。
  - **收益：** 减少不必要视觉推理、操作步数和布局脆弱性。
  - **风险：** API 权限更强，必须配套 safeguard 与 scope control 。

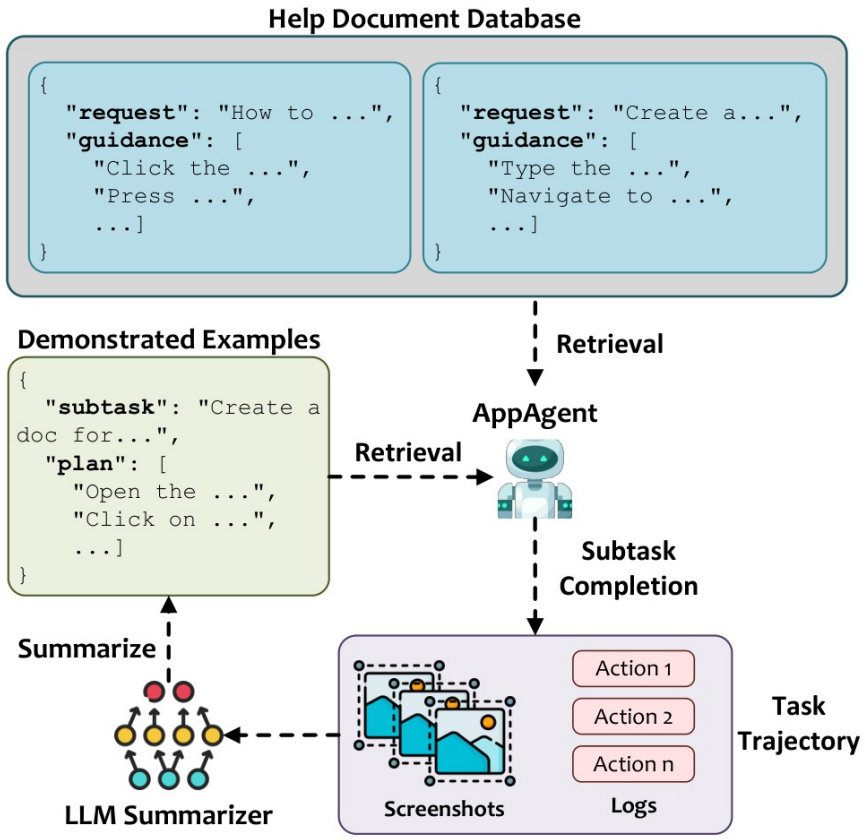
UIA 提供结构，视觉解析补足非标准控件

- **结构感知：** Windows UI Automation/Win32/WinCOM 提供 control metadata 与状态。
- **视觉感知：** OmniParser 等模型处理 canvas 、 custom widget 、无 accessibility 节点的界面。
- **融合：**先检索结构控件，再用视觉补足，形成统一 control list 。
- **持续知识：**应用文档、 patch note 、历史执行轨迹进入向量化 memory ，无需重新训练模型。



论文原图： UFO2 GUI/API hybrid

一次推理生成多个 GUI 动作，用轻量状态检查决定是否继续提交

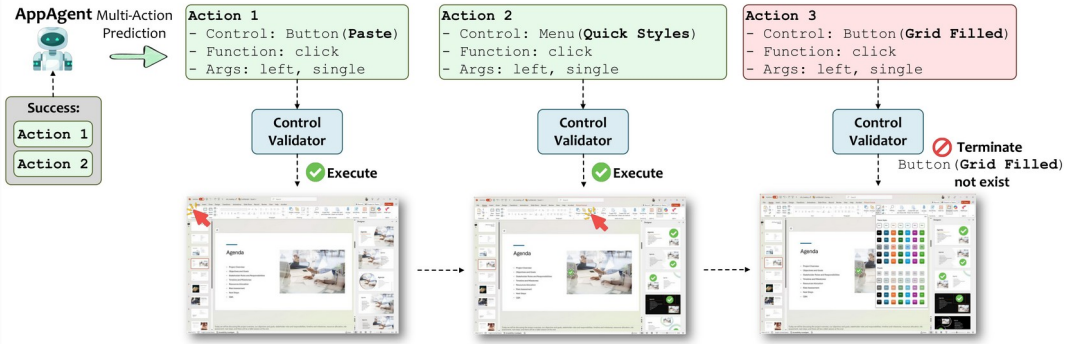


论文原图： UFO2 knowledge substrate

- **问题：** 每个 GUI step 都调用 LLM，延迟随轨迹长度线性累积。
- **方法：** 模型生成 multi-action sequence；执行引擎逐个提交动作。
- **验证：** 每个动作后检查 control/application state；若与预期一致则继续，异常则停止并重新规划。
- 与 Speculative Actions 的关系：都预测未来动作；UFO² 更依赖 OS control state，聚焦桌面 workflow。
- **风险：** 多动作越长，分支 / 副作用越难回滚；固定 horizon 不适合所有状态。

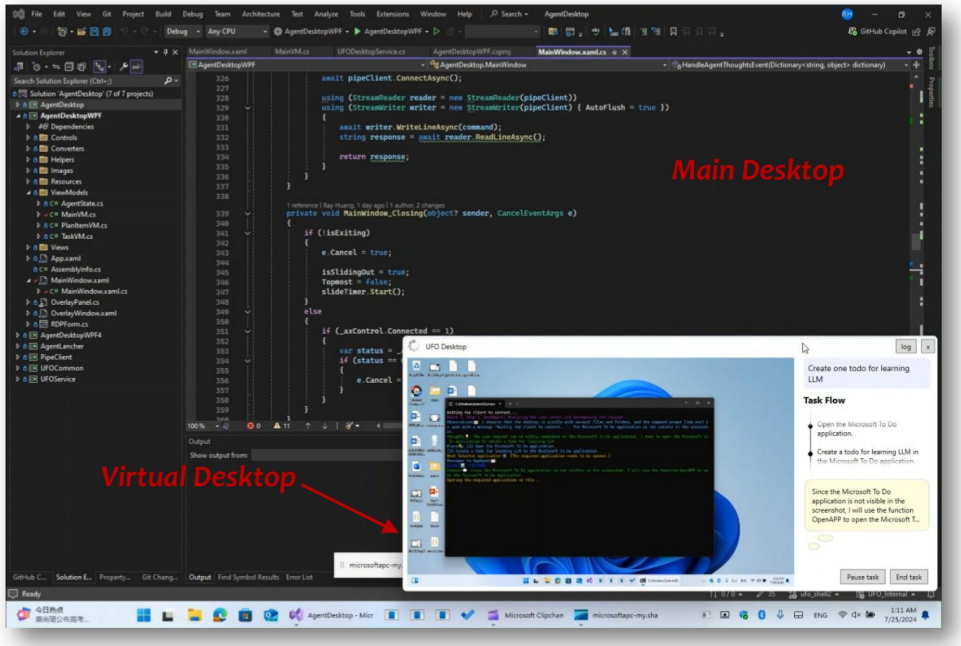
可靠执行还需要隔离用户桌面与高风险动作

- **PiP：** 基于 Windows remote desktop loopback 建立嵌套虚拟桌面；用户与 Agent 可并行工作。
- **Safeguard：** 对危险 / 越权操作进行检查，避免强 API 直接造成副作用。
- **实验：** 覆盖 20+ Windows apps；即使使用通用 GPT-4o，整体表现比专用 CUA 高超过 10%（论文报告）。
- **系统结论：** 深度 OS integration 的收益来自更强 observation/action substrate，而不只是更强模型。



论文原图： UFO2 speculative multi-action

移动 GUI 还要解决“每一步页面是否都能上传”

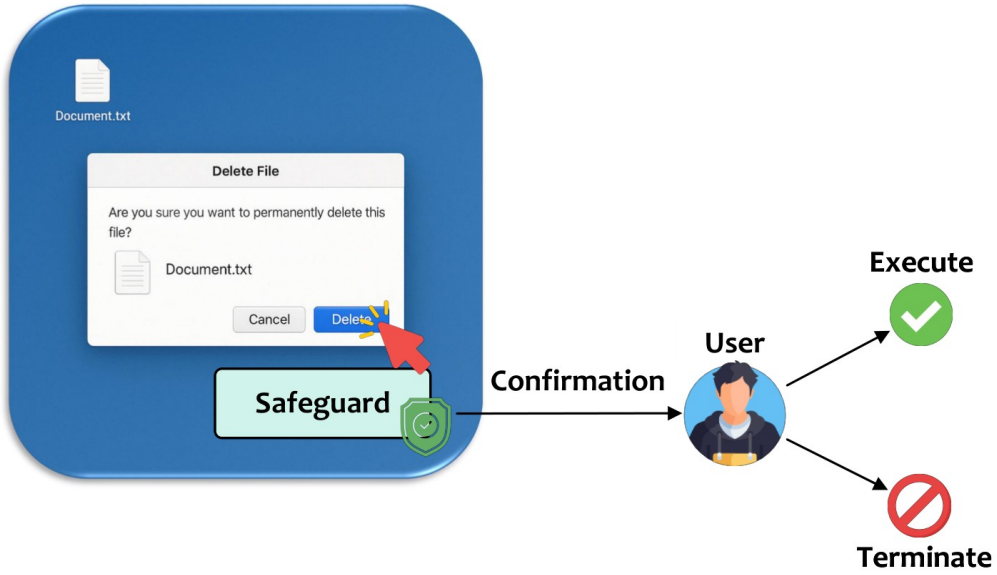


论文原图： UFO2 picture-in-picture

- **观察：** cloud mobile agent 通常每步上传完整 XML/screenshot；大部分 UI 与当前 action 无关。
- **CORE：** 本地 block partition/ranking + 云端 element decision + multi-round accumulation 。
- **执行意义：** 将 observation 的可见范围纳入 action pipeline，兼顾成功率与隐私。
- **结果：** UI exposure 最多减少 55.6%，成功率接近 cloud-only 。

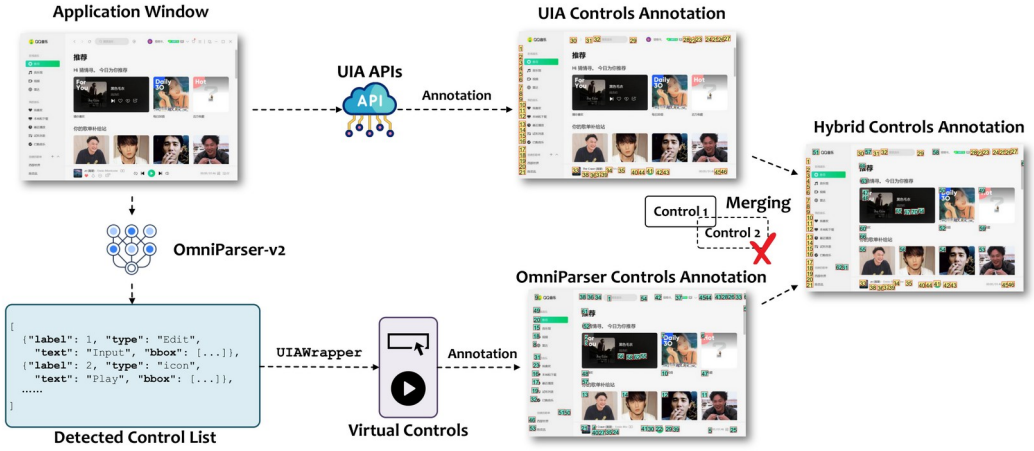
统一 desktop、 mobile、 browser、 tool/MCP 的 GUI Agent 模型谱系

- **GUI-Owl-1.5：** 2B/4B/8B/32B/235B； instruct 与 thinking variants。
- **数据：** 模拟环境 + cloud sandbox 的 hybrid data flywheel。
- **能力：** grounding、 GUI automation、 Tool/MCP、 memory、多 Agent adaptation。
- **训练：** 多平台 environment RL； MRPO 处理任务冲突与长轨迹低效问题。



论文原图： UFO2 safeguard

模型规模与平台统一，为端云协同提供可替换 Actor



论文原图： UFO2 control results

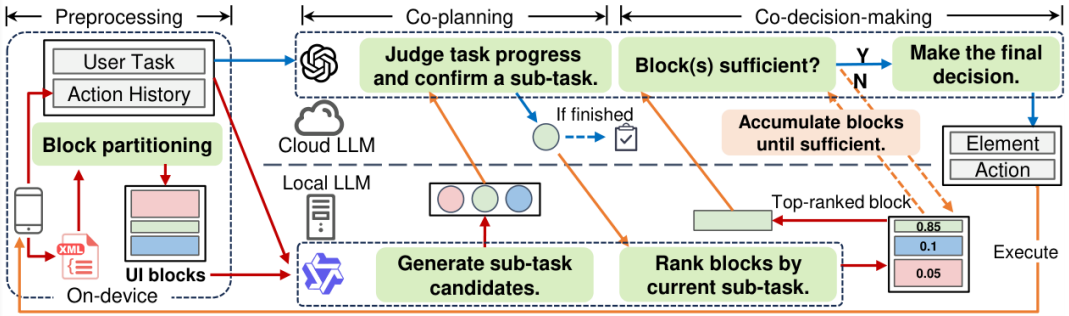
- **GUI automation：** OSWorld 56.5 、 AndroidWorld 71.6 、 WebArena 48.4 。
- **Grounding：** ScreenSpotPro 80.3 。
- **Tool calling：** OSWorld-MCP 47.6 、 MobileWorld 46.8 。
- **部署含义：**小尺寸模型承担低延迟常规步骤；大尺寸模型承担复杂规划与异常状态。
- **未解决：**榜单提升不自动带来失败检测、不可逆操作防护和完成条件证明。

>10%

vs dedicated CUA

同一个错误，应在三个不同时间点被发现

- **Pre-operation：** 动作尚未执行；判断 candidate 是否正确 / 危险—— GUI-Critic-R1 。
- **During/after each step：** 动作可能未生效；比对 observed effect 与 expected effect——VeriGUI 。
- **Post-trajectory：** 界面看似完成，但文件 / 配置 / 隐藏状态可能不对—— VAGEN 、 IRA 。
- **设计原则：**越早阻断副作用越小；越晚验证证据越完整但恢复成本越高。



论文原图： CORE pipeline

Look Before You Leap：在 click/delete/pay 之前先问“会发生什么”

- **输入 ε：** instruction + interaction history + current screenshot 。
- **输入 a：** Actor 生成的 candidate action 。
- **输出：** correctness score l∈[0,1] 、 critique c 、 corrective suggestion s 。

- **流程：** Critic 判断潜在 outcome ；正确则放行；错误则将 suggestion 返回 Actor 重规划。
- **优势：** 避免依赖执行后 undo ；适合不可逆、高副作用动作。

2B→235B
统一模型谱系

6
代表 benchmarks

reasoning bootstrapping + RFT cold start + suggestion-aware GRPO

- **数据：**从 AITZ、AMEX、Odyssey、AITW 收集正动作；用模型生成近分布负动作； GPT-4o 清洗。
- **Progressive CoT：** Observation → Possible Result → Critique → Score → Suggestion 。
- **Bootstrapping：**不直接泄漏 label/suggestion，采样多条思维，仅保留最终 score/suggestion 与标注一致者。

- **训练：** RFT 冷启动； S-GRPO 奖励 format、accuracy 与 suggestion quality 。
- **规模：**约 11K train，其中 6K 高质量 CoT；约 1K+ test，覆盖 mobile/web generalization 。

Pre-critic 将 AndroidWorld baseline success 由 22.4% 提升到 27.6%

- **静态：**专用 7B critic 在 GUI-Critic-Test 上优于通用开源 MLLM 。
- **动态：**发现错误、冗余和危险动作，并提供可执行修正建议。
- **收益：**阻止不可逆错误、缩短低效路径。

- **代价：**每一步增加 critic inference；错误拒绝会降低流畅性；训练使用 8×A100 。
- **向下一篇过渡：**即使 candidate 看起来正确，真实 GUI 也可能因延迟 / 弹窗而未执行成功。

22.4→27.6
AndroidWorld success

Thinking–Verification–Action–Expectation （TVAE）把状态转移显式化

- **Thinking：** 基于目标和历史判断当前意图。
- **Verification：** 检查上一步是否产生预期效果；识别 unchanged screen 、错误页面或干扰。
- **Action：** 若成功则推进；若失败则选择 corrective action ，而不是重复点击。

- **Expectation：** 预测当前 action 应产生的下一状态，为下一步验证提供参照。
- **训练：** Robust SFT 注入 synthetic failure trajectories ； GRPO 使用 asymmetric verification reward 。

常规 step accuracy 会掩盖“是否识别失败并恢复”

- **新指标：** Loop Rate （重复失败动作比例）、 Recovery Success Rate （正确恢复比例）、 Sim-TSR 、 ASO 。
- **结果：** VeriGUI-3B/7B RSR 51.1%/52.5% ，为测试开源模型最佳。

- **VeriGUI-7B：** Sim-TSR 23.5% ， ASO 1.09 。
- **观察：** 标准 SFT 可提高动作准确率，却可能让错误动作更自信；显式 failure training 才提高 recovery 。

51.1/52.5%	23.5%	1.09
RSR 3B/7B	Sim-TSR	ASO

可见界面只是环境状态的一部分

- **可见状态：**按钮、文本、窗口、页面。
- **隐藏状态：**系统设置、应用配置、数据库、邮件发送状态、文件内容、生成 artifact 。

- **被动 judge 的失败：**从最后截图推断“已完成”，可能遗漏隐藏错误； Actor 的文字解释还可能反向误导 evaluator 。
- **新范式：** Verifier 也要像 Agent 一样主动调用工具取得证据。

Progressive Verification： surface→latent， cheap→expensive

1 Step 1

整理 Actor trajectory，仅保留可核验的操作与视觉证据。

2 Step 2

优先检查已有 screenshots/observations；证据充分则 early stop。

3 Step 3

证据不足时调用 shell、Python 或 GUI tool probe 环境。

4 Step 4

综合证据判断 task success，作为 RL reward 或 test-time selection 信号。

- **结果：** OSWorld 人工真值下 Acc 92.9%/93.4%；用于 GRPO 后 success 34.3%，比最强 baseline 高 3.8pp。

92.9/93.4%	34.3%	+3.8pp
verification accuracy	OSWorld success	vs baseline

Propose-then-Verify 将“任务完成”拆成可检查证据

1 Step 1

根据 instruction 提出 completion conditions。

2 Step 2

为每个条件选择 system、application 或 GUI tool。

3 Step 3

读取配置、文件 / artifact 或界面状态，形成 evidence。

4 Step 4

聚合各条件，输出 final judgement。

- **GUI-RewardBench：** 321 trajectories、10 类 Ubuntu apps，覆盖 visible/hidden/artifact verification。
- **结果：** 86.9% accuracy；作为 RL reward 得到 34.0% OSWorld success，接近 script reward 34.9%。

86.9%	34.0%	34.9%
GUI-RewardBench	OSWorld success	script reward